# GPU Micromagnetic Simulation with Irregular Geometry

Qianru Wu
Carnegie Mellon University

March 2026

We use multiple problem sizes in different sections to highlight different performance regimes, including overhead-dominated and bandwidth-dominated behavior.

## Project URL

Project page link: https://angelawoo572.github.io/gpu-micromagnetics-irregular/

## Summary

We study the end-to-end performance of a GPU-accelerated micromagnetic simulation for the Landau–Lifshitz–Gilbert (LLG) equation using CUDA and SUNDIALS/CVODE. Although each right-hand side (RHS) evaluation is data-parallel across grid cells, the full simulation is controlled by an adaptive implicit time-integration pipeline, introducing sequential dependencies and solver-side orchestration.

We first analyze a regular-grid local-interaction solver through layout, block configuration, tolerance, scaling, and Nsight profiling experiments. These results show that kernel-level optimizations provide only modest improvements, while synchronization, solver control flow, and repeated SUNDIALS vector operations dominate runtime. We then extend the solver to irregular masked geometries and compare dense masked execution with compact active-cell execution, showing that reducing parallel work alone yields limited gains. Finally, we integrate a GPU-resident FFT-based demagnetization path, replacing an $O(N^2)$ convolution with an $O(N \log N)$ method.

Overall, performance is determined by the interaction between GPU kernels and the CVODE-driven solver pipeline, rather than by the efficiency of any single kernel.

## Introduction

We study the performance of GPU-accelerated micromagnetic simulation for the Landau–Lifshitz–Gilbert (LLG) equation, with the goal of identifying which components of the system determine end-to-end runtime.

The LLG equation describes the time evolution of magnetization vectors on a spatial grid. After discretization, the system has three unknowns per cell. Each RHS evaluation combines local stencil-based interactions, material-dependent local terms, and a long-range demagnetization field. The local part is naturally data-parallel across grid cells, while the demagnetization term introduces global coupling.

However, time integration is performed using CVODE, an adaptive implicit solver that introduces a hierarchy of sequential operations, including Newton iterations and Krylov linear solves. These steps require repeated vector operations, reductions, and synchronization, which may limit the benefits of GPU parallelism.

To understand this interaction between data-parallel kernels and solver-driven control flow, we proceed in three stages. We first analyze a regular-grid local-interaction solver to establish baseline performance and identify bottlenecks. We then introduce irregular masked geometries to study how nonuniform domains affect both GPU execution and solver behavior. Finally, we implement solver-level optimizations targeting synchronization and linear-algebra costs, and integrate an FFT-based demagnetization path for long-range interactions.

The experiments are organized to show where the bottleneck moves as the implementation grows from a local regular-grid solver to irregular geometries and then to FFT-based long-range demagnetization.

## System Overview

**Problem Structure and Parallelism.** The simulation evolves a magnetization field on a 2D grid with three components per cell, stored in a structure-of-arrays (SoA) layout. Each RHS evaluation computes an effective field consisting of

- local stencil-based interactions (nearest-neighbor exchange and related terms),
- per-cell anisotropy and other local contributions,
- and a global demagnetization field.

The local terms follow a regular stencil pattern and are computed with one CUDA thread per grid cell, exposing straightforward data parallelism. In contrast, the demagnetization field introduces global coupling and is implemented using FFT-based convolution.

Time integration is performed by CVODE, which repeatedly invokes the RHS inside an adaptive implicit solver. As a result, the overall execution is not purely data-parallel: it includes many sequential solver steps, vector operations, and synchronization points that interact with GPU kernel execution.

**Three-Layer Performance Architecture.** The implementation is organized into three layers corresponding to distinct performance concerns. Each layer targets a different source of cost in the overall pipeline.

Table 1: The three optimization layers and their corresponding GPU strategies.

| **Layer** | **Concrete optimization in this code** |
|---|---|
| Local kernel | Tiled shared-memory stencil; fused window-and-gather; branch-free normalization; `__ldg` read-only cache; `__launch_bounds__`; SoA layout (*memory reuse, divergence control, stencil efficiency*) |
| Long-range | $O(N^2) \to O(N \log N)$ via batched cuFFT; D2Z/Z2D half-spectrum using Hermitian symmetry; device-resident tensor $\hat{\mathbf{F}}$ (*algorithmic acceleration, reduced memory traffic*) |
| Implicit solver | Custom `nvdotprodmulti` (multi-dot); analytic $\mathbf{Jv}$; block-Jacobi $3 \times 3$ preconditioner; CUDA streams (*reduction fusion, synchronization reduction*) |

**Key Insight.** These three layers isolate local computation, long-range interaction, and solver orchestration. As the following sections show, optimizing any single layer in isolation yields limited improvement; overall performance is determined by how these components interact within the CVODE-driven pipeline.

## Baseline End-to-End Analysis on the Regular Grid

All experiments in this section use a baseline grid size of $n_x = 900$, $n_y = 128$ unless otherwise specified. We use this regular-grid case to separate three effects: data layout, thread-block shape, and solver tolerance. All results report total **GPU time-to-solution**, since CVODE introduces sequential dependencies across timesteps.

### Kernel-Level Tuning: Layout and Block Shape

We first test two standard CUDA-level choices: data layout and thread-block configuration.

**Layout: AoS vs. SoA.** We compare:

- **AoS (Array of Structures):** $(m_x, m_y, m_z)$ stored per cell
- **SoA (Structure of Arrays):** separate arrays for $m_x, m_y, m_z$

| Layout | Time (ms) | Speedup |
|---|---|---|
| AoS | $\sim 14796$ | 1.00x |
| SoA | $\sim 14733$ | $\sim 1.004$x |

SoA is slightly faster, but the gain is small because each thread still needs all three magnetization components.

**Block shape.** We sweep thread-block shapes and find that **BLOCK_X = 16, BLOCK_Y = 8** performs best. Other configurations produce measurable differences, but only within a few percent. These two experiments establish that conventional kernel-level tuning is useful, but not large enough to explain the end-to-end runtime.

### Solver-Level Sensitivity: Tolerance

We next vary the CVODE tolerance, which changes the number of accepted steps, failed nonlinear solves, and error-test failures.

| Tolerance | Time (ms) | ncfn | netf |
|---|---|---|---|
| $10^{-4}$ | **13415** | 553 | 1 |
| $10^{-5}$ | 14560 | 0 | 1 |
| $10^{-6}$ | 13934 | 558 | 6 |

The fastest setting is $10^{-4}$, while $10^{-5}$ is the most robust because it eliminates nonlinear convergence failures. Tightening to $10^{-6}$ does not improve runtime. Compared with layout or block shape, tolerance has a larger effect on total time, so later experiments use $10^{-4}$ when the goal is performance.

### Problem-Size Scaling and Nsight Analysis

Using SoA layout, $16 \times 8$ blocks, and tolerance $10^{-4}$, we vary problem size by increasing $n_x$ while holding $n_y = 128$ fixed.

| $n_x$ | Total cells | Time (ms) |
|---|---|---|
| 96 | 4096 | 6823.372 |
| 192 | 8192 | 7032.230 |
| 384 | 16384 | 7100.213 |
| 768 | 32768 | 7795.561 |
| 1536 | 65536 | 8990.308 |
| 3072 | 131072 | 9941.907 |
| 6144 | 262144 | 11180.528 |
| 12288 | 524288 | 16812.521 |

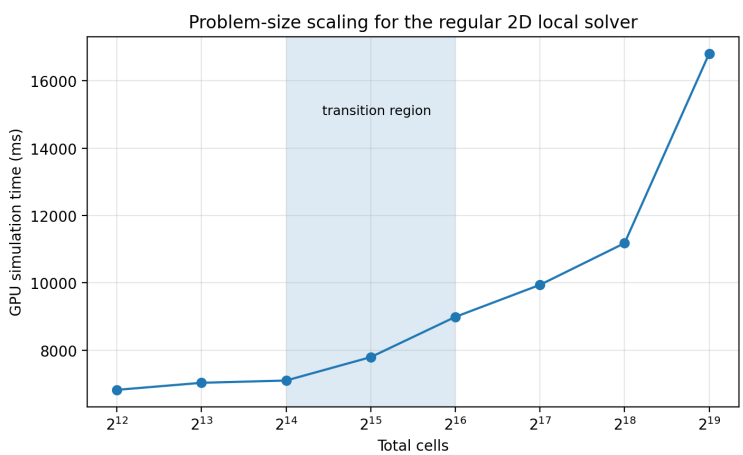


Figure 1: Problem-size scaling of the regular 2D local solver. The smallest cases are nearly flat, indicating fixed overhead dominance. Runtime increases more clearly beyond the small-size regime, with a transition around $1.6 \times 10^4$ to $6.6 \times 10^4$ cells.

The first three cases are almost flat: increasing the problem from 4096 to 16384 cells changes runtime from 6.82 s to 7.10 s, only about a 4% increase while the number of cells grows by $4\times$. This is the expected behavior when fixed costs such as kernel launches, synchronization, and solver bookkeeping dominate arithmetic work.

Beyond this range, runtime grows more clearly, reaching 16.8 s at 524288 cells. This separates the regular-grid solver into three regimes:

- **Small:** fixed overhead dominates.
- **Transition:** overhead and GPU computation both matter.
- **Large:** the per-step GPU workload becomes more visible.

This matches the standard GPU performance model: small data-parallel workloads often fail to amortize launch and synchronization overhead, while larger workloads expose more of the device computation and memory traffic.

To identify the overhead directly, we profile three representative sizes with Nsight Systems:

- **Small:** 4096 cells
- **Mid:** 65536 cells
- **Large:** 524288 cells

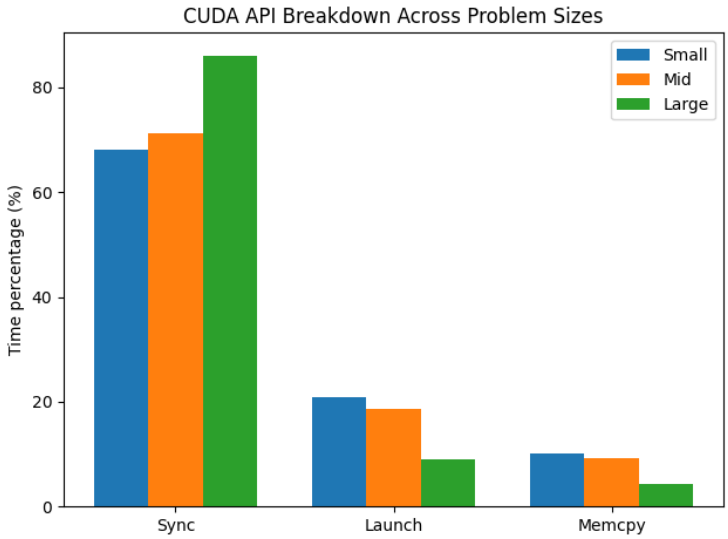


Figure 2: CUDA API time breakdown for small, mid, and large problem sizes. Synchronization dominates across all sizes, while kernel launch and memory copy become relatively less important at larger scales.

| Metric | Small | Mid | Large |
|---|---|---|---|
| cudaStreamSynchronize | 68.1% | 71.2% | 86.1% |
| cudaLaunchKernel | 20.9% | 18.7% | 9.0% |
| cudaMemcpyAsync | 10.1% | 9.3% | 4.4% |
| RHS kernel share | 7.2% | 10.4% | 12.1% |

Across all three sizes, `cudaStreamSynchronize` is the dominant CUDA API cost. Its share increases with problem size, reaching 86.1% in the largest case. The RHS kernel remains a minority of GPU kernel time, rising only from 7.2% to 12.1%. Thus the regular-grid bottleneck is not a single RHS kernel, but the CVODE-controlled sequence of synchronization points and SUNDIALS vector operations around it.

**Conclusion.** The regular-grid study establishes the baseline used in later sections: SoA layout, $16 \times 8$ blocks, and tolerance $10^{-4}$. This configuration prioritizes speed over robustness, since the fastest tolerance still has nonzero nonlinear convergence failures. More importantly, these experiments show why kernel-level tuning alone is not enough: as problem size changes, runtime is largely determined by how much CVODE overhead is amortized by the GPU workload.

## Irregular Geometry as a Systems Stress Test

We next extend the regular 2D local solver to an irregular masked geometry embedded in the same structured grid. Unless otherwise specified, these experiments use the periodic 2D code path with a circular interior hole and a baseline problem size of $n_x = 1536$, $n_y = 128$.

### Dense Masked Baseline

The irregular domain uses the same periodic grid in $x$ and $y$, so the outer domain is topologically a torus. Irregularity is introduced by masking out a circular hole in the interior; only the remaining active cells evolve.

Our first implementation is a correctness-first dense masked baseline. The state remains in SoA layout,

$$[m_x \text{ for all cells}][m_y \text{ for all cells}][m_z \text{ for all cells}].$$

The execution policy is:

- launch one CUDA thread per physical cell in the full periodic grid,
- set the derivative to zero if the current cell is inactive,
- skip neighbor contributions from inactive cells,
- use periodic wrap-around for neighbor indexing in both dimensions.

This preserves the original CVODE + CUDA structure and isolates the effect of irregular geometry before introducing a different execution strategy.

As a sanity check, we run the same irregular code path with `HOLE_RADIUS_FRAC_Y = 0.0`, so the entire domain remains active.

| Case | Active Cells | Time (ms) | nfe | ncfn |
|---|---|---|---|---|
| Periodic regular ($R = 0$) | 65536 / 65536 | 4425.310 | 4021 | 207 |

This gives a periodic baseline from the irregular code path itself. We then vary the circular hole size while keeping dense full-grid execution.

| Hole Radius Fraction | Active % | Time (ms) | nst | nfe | ncfn |
|---|---|---|---|---|---|
| 0.10 | 99.20% | 8234.460 | 6816 | 8015 | 580 |
| 0.16 | 98.01% | 8594.956 | 6819 | 8028 | 584 |
| 0.22 | 96.19% | 8789.738 | 6920 | 8344 | 692 |
| 0.28 | 93.83% | 8855.327 | 6551 | 7760 | 2 |
| 0.34 | 90.94% | 9312.673 | 6556 | 7784 | 2 |

Introducing even a modest hole substantially increases end-to-end runtime relative to the $R = 0$ case. Runtime also does not scale simply with the number of active cells: larger holes do not automatically produce less work or shorter runtime. The hole changes the local coupling near the boundary, which changes CVODE step counts, RHS evaluation counts, and nonlinear convergence behavior. In this setting, irregular geometry is both an execution issue and a numerical solver issue.

### Compacted Active-Cell Execution

To reduce wasted work on inactive cells, we implemented a compact active-cell version:

- the global state vector remains in the dense SoA layout expected by CVODE,
- an explicit active-cell index list is built,
- the main irregular RHS kernel launches only over active cells,
- inactive entries remain in the state vector and are explicitly zeroed in $\dot{y}$.

This compares two execution strategies on the same geometry:

- **Dense masked execution:** full-grid launch with inactive-cell checks.
- **Compact active-cell execution:** launch only over active cells.

For a circular hole with radius fraction 0.22:

| Execution Strategy | Active % | Time (ms) | nst | nfe | ncfn |
|---|---|---|---|---|---|
| Dense masked | 96.19% | 8789.738 | 6920 | 8344 | 692 |
| Compact active-cell | 96.19% | **8464.103** | 6921 | 8347 | 693 |

The end-to-end improvement is

$$\frac{8789.738 - 8464.103}{8789.738} \approx 3.7\%.$$

The CVODE statistics are almost unchanged between the two versions, so the runtime difference is mainly due to execution organization rather than a different solver path. Compaction avoids inactive threads, but the inactive fraction is only 3.81%; it also introduces active-index indirection and still requires a kernel to zero inactive entries of $\dot{y}$. The result is a real but modest improvement.

### What Irregular Geometry Reveals

Nsight Systems explains why the compact version improves total runtime only slightly. For the dense masked case with $R = 0.22$, the irregular RHS kernel averages $6.92\,\mu s$ per launch. In the compact version, the compact RHS kernel averages $7.19\,\mu s$, and the inactive-zeroing kernel adds another cost. Per launch, the compact RHS is slightly slower because of active-index indirection and extra integer reconstruction.

Compaction still improves end-to-end time because it avoids applying the main physics kernel to inactive cells. However, profiling shows that the physics RHS is not the dominant cost. In the compact run, `cudaStreamSynchronize` accounts for roughly 71.6% of CUDA API time, while the compact RHS kernel is only about 12.7% of total GPU kernel time. SUNDIALS vector kernels such as `linearSumKernel`, `dotProdKernel`, `scaleKernel`, and `wL2NormSquareKernel` together account for a larger share.

**Conclusion.** The irregular experiments separate two effects. Dense masking is simple and preserves regular memory access, but wastes work on inactive cells. Compact execution removes some wasted work, but the benefit is limited when the inactive fraction is small and the solver pipeline remains the dominant cost. This is why irregular geometry changes the workload, but does not change the main bottleneck: the dominant cost remains outside the physics kernel itself.

## Solver-Level Optimizations

The regular-grid and irregular-geometry studies showed that the RHS kernel was not the only important cost. We therefore optimized the CVODE/SPGMR layer, where repeated vector operations, reductions, and synchronization dominate the execution trace.

Unless otherwise specified, this section uses a large regular grid with $n_x = 3000$, $n_y = 1280$, $neq = 3{,}840{,}000$, and tolerance $10^{-4}$. Timings are from Nsight Systems profiling runs, which were more stable than bare `make run` measurements.

### Large-Grid Baseline

| Configuration | Time (ms) | `cudaStreamSync` share |
|---|---|---|
| Baseline (SoA, 16×8, tol $10^{-4}$) | 46,249 | ∼44.1% |

In this baseline, `f_kernel_group_soa_periodic` accounts for only 8.9% of GPU kernel time. Most GPU time is instead spent in SUNDIALS vector kernels such as `linearSumKernel`, `dotProdKernel`, and `wL2NormSquareKernel`. On the CUDA API side, `cudaStreamSynchronize` accounts for about 44%, with additional cost from `cudaLaunchKernel`, `cudaMemcpyAsync`, and initialization calls such as `cudaMalloc`. This baseline motivates optimizing the CVODE/SPGMR orchestration layer rather than only the RHS kernel.

### Optimization 1: Fused NVector and Multi-Dot

**Motivation.** In CVODE's default immediate-execution model, many vector operations launch separate CUDA kernels and synchronize when scalar results are needed. In the small-grid baseline, the full simulation performs 291,455 synchronizations.

**Design.** We implement a fused NVector layer in `deferred_nvector.cu`. Tier 1 enables SUNDIALS' built-in fused CUDA operations with `N_VEnableFusedOps_Cuda(v, 1)`. Tier 3 overrides `nvdotprodmulti` with a custom `multi_dot_kernel`. During SPGMR Classical Gram–Schmidt (CGS), the $K$ dot products against the current Krylov basis are independent. The custom kernel reads the reference vector $x$ once, accumulates against all $K$ basis vectors in shared memory, and produces $K$ scalars with one kernel launch and one `cudaStreamSynchronize`, instead of $K$ separate reductions.

```cpp
__global__ static void multi_dot_kernel(
    const sunrealtype* __restrict__ x,
    FusedPtrBundle Y,
    sunrealtype*   __restrict__ out,
    sunindextype n, int K)
{
    extern __shared__ sunrealtype smem[];   // K * blockDim.x doubles
    const int tid = threadIdx.x, bsz = blockDim.x;

    for (int k=0; k<K; ++k) smem[k*bsz + tid] = 0.0;
    __syncthreads();

    for (sunindextype i = blockIdx.x*bsz + tid; i < n; i += gridDim.x*bsz) {
        const sunrealtype xi = x[i];
        for (int k=0; k<K; ++k)
            smem[k*bsz + tid] += xi * Y.p[k][i];
    }
```

```
    __syncthreads();

    for (int s = bsz>>1; s > 0; s >>= 1) {
        if (tid < s)
            for (int k=0; k<K; ++k)
                smem[k*bsz + tid] += smem[k*bsz + tid + s];
        __syncthreads();
    }
    if (tid == 0)
        for (int k=0; k<K; ++k) atomicAdd(&out[k], smem[k*bsz]);
}
```

Listing 1: `multi_dot_kernel`: $K$ dot products in one launch and one synchronization.

Pointer and coefficient arrays are passed as `struct`-typed kernel arguments, using 128 bytes in the 4 KB per-launch constant buffer. An earlier implementation copied these small arrays with `cudaMemcpy`; it added about 1.6 s of synchronous transfer time and was discarded. We also override `nvclone`, so vectors allocated internally by CVODE, including Krylov basis vectors and error/weight vectors, inherit the optimized operations.

**Failure mode on the large grid.** CGS triggers `N_VDotProdMulti`, but its correction step also calls `N_VLinearCombination`, which reads all $K$ Krylov vectors simultaneously. For $neq = 3.84 \times 10^6$, each Krylov vector is 30 MB. Reading five vectors at once exceeds the L2 cache and reduces effective bandwidth from 88% to 17%. With the `linear_comb_kernel` override, the large-grid run regressed badly:

| Configuration | Time (ms) | vs. baseline |
|---|---|---|
| Baseline (large grid) | 46,249 | 1.00× |
| Fused + CGS + `linear_comb_kernel` | 103,321 | 0.45× |

The `linear_comb_kernel` alone consumed 54.8 s, more than the entire baseline runtime. The saved synchronizations were outweighed by cache thrashing and extra memory traffic.

**Adaptive GS selection.** We therefore select the Gram–Schmidt strategy by problem size:

- $neq < 500{,}000$: use Classical GS and install the `linear_comb_kernel` override. These runs are overhead-limited, so reducing synchronization helps.
- $neq \geq 500{,}000$: use Modified GS and skip the linear-combination override. These runs are bandwidth-limited, so reading many full vectors at once hurts more than the saved launches help.

**Small-grid result.** For a $600 \times 128$ grid with $neq = 76{,}800$:

| Configuration | Time (ms) | Speedup |
|---|---|---|
| Baseline (small grid) | 5,348 | 1.00× |
| Fused NVector + CGS | 847 | **6.3**× |

The synchronization count falls by 43%, and runtime improves by 6.3×. For the large-grid regime, the fused layer is evaluated only as part of the combined solver-level configuration below.

### Optimization 2: Analytic $3 \times 3$ Block-Jacobi Preconditioner

**Motivation.** Without preconditioning, SPGMR requires about 5.5 GMRES iterations per Newton step. Each iteration performs vector operations such as `linearSumKernel`, `dotProdKernel`, and `wL2NormSquareKernel`; GMRES-related work accounts for about 60% of GPU kernel time.

**Design.** For each cell $i$, we derive a $3 \times 3$ approximation to $J_{ii} = \partial f_i / \partial m_i$, holding neighbor magnetizations fixed. The only self-dependent effective-field term is the easy-axis anisotropy, $\partial h_3 / \partial m_3 = c_{\text{chk}} = 1$. The local block is:

$$
\begin{aligned}
&J_{00} = c_\alpha(-h_1 m_1 - \mu), \quad J_{01} = -c_{\text{chg}} h_3 - c_\alpha h_2 m_1, \quad J_{02} = c_{\text{chg}}(h_2 - m_2) - c_\alpha(h_3 + m_3) m_1 \\
&J_{10} = c_{\text{chg}} h_3 - c_\alpha h_1 m_2, \quad J_{11} = c_\alpha(-h_2 m_2 - \mu), \quad J_{12} = c_{\text{chg}}(m_1 - h_1) - c_\alpha(h_3 + m_3) m_2 \\
&J_{20} = -c_{\text{chg}} h_2 - c_\alpha h_1 m_3, \quad J_{21} = c_{\text{chg}} h_1 - c_\alpha h_2 m_3, \quad J_{22} = c_\alpha(1 - (h_3 + m_3) m_3 - \mu)
\end{aligned}
$$

where $\mu = m \cdot H$. This captures the precession cross-terms $J_{01} = -c_{\text{chg}} h_3$ and $J_{10} = +c_{\text{chg}} h_3$, which scalar Jacobi would miss. We form $P_i = I - \gamma J_i$, invert it analytically using Cramer's rule, and store $P_i^{-1}$ as 9 doubles per cell, about 87 MB total. The `psolve_kernel` applies $z = P^{-1} r$ as one batched $3 \times 3$ matrix-vector multiply per cell.

**Bug found by profiling.** The first version returned immediately when CVODE passed `jok=SUNTRUE`, reusing the previous $P^{-1}$. Nsight showed:

| Kernel | Calls | Expected |
|---|---|---|
| `psetup_kernel` (v1) | 266 | ≈3,753 |
| `psolve_kernel` | 20,207 | – |

CVODE's `jok` flag only means that the Jacobian structure can be reused; it does not mean that $\gamma$ is unchanged. Since $P = I - \gamma J$, every Newton step can change $P^{-1}$. Reusing a stale inverse made the preconditioner inconsistent with the current linear system.

The corrected version separates Jacobian storage from inverse construction:

- `build_J_kernel`: called only when `jok=SUNFALSE`; reads $y$ and four neighbors, computes $J_i$, and stores 9 doubles per cell. Its cost is comparable to one RHS evaluation.
- `build_Pinv_kernel`: called on every `psetup`; reads $d_J$ and the current $\gamma$, then computes $P_i^{-1}$. This is arithmetic-only, with no stencil reads.

**Result and limitation.** The corrected preconditioner is reported as part of the final combined solver configuration:

| Configuration | Time (ms) | Speedup vs. baseline |
|---|---|---|
| Baseline | 46,249 | 1.00× |
| Fused + Precond v2 + JTV | 24,901 | 1.86× |

The block-Jacobi preconditioner does not substantially reduce the GMRES iteration count, which remains about 5.5 iterations per Newton step. The reason is structural: the preconditioner

captures only self-coupling, while the dominant off-diagonal Jacobian terms come from exchange coupling to the four neighbor cells,

$$\partial H_i/\partial m_j = c_{\text{che}} = 4.0,$$

four times larger than the anisotropy self-term and excluded by construction. Thus the preconditioner changes the cost of the linear solve more than it changes the outer integration path.

### Optimization 3: Analytic Jacobian–Vector Product

**Motivation.** With Fused NVector and the first preconditioner version active, Nsight reports 20,489 calls to `f_kernel_group_soa_periodic`, while CVODE reports only `nfe`=3,756 true RHS evaluations. The extra ≈ 16,700 calls come from the default finite-difference Jacobian-vector product:

$$J(t, y)v \approx \frac{f(y + \varepsilon v) - f(y)}{\varepsilon}.$$

At about 85 $\mu$s per RHS kernel, these extra evaluations contribute roughly 16,700 × 85 $\mu$s ≈ 1.42 s.

**Design.** We register `JtvProduct` with `CVodeSetJacTimes(cvode_mem, NULL, JtvProduct)`. For perturbation $v$, the field variation is

$$\begin{aligned}
\delta h_1^{(i)} &= (c_{\text{che}} + c_{\text{chb}})(v_{1,L} + v_{1,R}) + c_{\text{che}}(v_{1,U} + v_{1,D}) \\
\delta h_2^{(i)} &= c_{\text{che}}(v_{2,L} + v_{2,R} + v_{2,U} + v_{2,D}) \\
\delta h_3^{(i)} &= c_{\text{che}}(v_{3,L} + v_{3,R} + v_{3,U} + v_{3,D}) + v_{3,i}
\end{aligned}$$

and

$$\begin{aligned}
(Jv)_{1,i} &= c_{\text{chg}}(v_{3,i}h_2 + m_3\,\delta h_2 - v_{2,i}h_3 - m_2\,\delta h_3) + c_\alpha(\delta h_1 - \delta\mu\, m_1 - \mu\, v_{1,i}) \\
(Jv)_{2,i} &= c_{\text{chg}}(v_{1,i}h_3 + m_1\,\delta h_3 - v_{3,i}h_1 - m_3\,\delta h_1) + c_\alpha(\delta h_2 - \delta\mu\, m_2 - \mu\, v_{2,i}) \\
(Jv)_{3,i} &= c_{\text{chg}}(v_{2,i}h_1 + m_2\,\delta h_1 - v_{1,i}h_2 - m_1\,\delta h_2) + c_\alpha(\delta h_3 - \delta\mu\, m_3 - \mu\, v_{3,i})
\end{aligned}$$

with $\mu = m \cdot H$ and $\delta\mu = v \cdot H + m \cdot \delta H$. This has the same 4-neighbor stencil structure as the RHS kernel and is evaluated in one CUDA kernel launch, with no extra $f()$ call and no host transfer.

**Result.** JTV is evaluated together with the corrected preconditioner:

| Configuration | `f_kernel` calls | `jtv_kernel` calls | Time (ms) |
|---|---|---|---|
| Pre-JTV configuration | 20,489 | – | 28,440 |
| Fused + Precond v2 + JTV | 3,655 | 16,957 | 24,901 |

The RHS call count drops from 20,489 to 3,655, matching `nfe`; the finite-difference RHS evaluations are eliminated. The 16,957 `jtv_kernel` calls replace those evaluations. Each JTV call costs about 127 $\mu$s, compared with about 85 $\mu$s for the RHS kernel, consistent with the extra reads required for the perturbation vector.

The outer solver statistics are almost unchanged:

| Configuration | nst | nfe | nsetups | nni | ncfn | netf |
|---|---|---|---|---|---|---|
| Fused + Precond v1 (jok bug; FD Jv) | 3027 | 3756 | 523 | 3753 | 270 | 2 |
| Fused + Precond v2 + JTV | 3027 | 3756 | 155 | 3753 | 270 | 2 |

JTV therefore changes the cost of the linear solve, not the integration trajectory. Its main effect is to replace finite-difference RHS evaluations with a direct analytic kernel.

### Cumulative Effect and Remaining Bottleneck

| Configuration | Time (ms) | Speedup |
|---|---|---|
| Baseline (large grid) | 46,249 | 1.00× |
| Fused + CGS on large grid (*regression*) | 103,321 | 0.45× |
| Fused (adaptive) + Precond v2 + JTV | **24,901** | **1.86**× |
| *Small grid (600×128, neq = 76,800):* | | |
| Baseline | 5,348 | 1.00× |
| Fused + Precond v2 + JTV | 847 | **6.3**× |

The CGS regression is a useful negative result: reducing synchronization can increase runtime when the added kernel performs bandwidth-heavy multi-vector reads. This is why the final implementation selects the GS strategy by problem size.

**Final kernel breakdown.** For the final large-grid configuration:

| Kernel | GPU time share | Instances | Avg. ($\mu$s) |
|---|---|---|---|
| `psolve_kernel` | 19.5% | 20,693 | 226 |
| `linearSumKernel` (SUNDIALS) | 16.6% | 101,032 | 39 |
| `dotProdKernel` (SUNDIALS) | 13.9% | 86,294 | 39 |
| `linearCombinationKernel` (SUNDIALS) | 11.9% | 13,719 | 208 |
| `jtv_kernel` | 9.0% | 16,957 | 127 |
| `prodKernel` (SUNDIALS) | 7.1% | 20,693 | 82 |
| `f_kernel_group_soa_periodic` | 1.3% | 3,655 | 86 |

The four GMRES-related kernels in the first four rows account for about 62% of GPU time. GMRES still averages

$$20{,}693/3{,}753 \approx 5.5$$

iterations per Newton step, the same as the unpreconditioned baseline. Capturing the missing exchange coupling would require a spatial preconditioner such as additive Schwarz over patches, but dense block storage at practical patch sizes ($\geq 4 \times 4$ cells) exceeds available GPU memory for the problem sizes tested.

These observations highlight two points that carry over to the FFT experiments. Reducing kernel launches or synchronizations is not always beneficial: the CGS regression shows that fusion can lose when it increases memory traffic. In addition, a kernel that appears high in the profile is not necessarily a bad kernel; JTV and preconditioning add visible kernels because they replace more expensive solver work elsewhere.

**Conclusion.** The final solver-level configuration gives a 1.86× speedup on the large grid and a 6.3× speedup on the small grid. The three optimizations affect different costs: fused NVector reduces synchronization, the preconditioner changes the linear-solve work, and analytic JTV removes finite-difference RHS calls. The large-grid CGS regression is the main cautionary result: an optimization that helps in an overhead-limited regime can become harmful once the problem becomes bandwidth-limited.

## FFT-Based Demagnetization

The local solver above computes exchange, anisotropy, and other nearest-neighbor terms with stencil-like CUDA kernels. We next add the long-range demagnetization field. Unlike the local terms, demagnetization is globally coupled: the field at one target cell depends on magnetization values at all source cells. In direct form,

$$h_i = \sum_j f_{ij} y_j,$$

where $y_j$ is the magnetization at source cell $j$, $f_{ij}$ is the demagnetization interaction tensor, and $h_i$ is the field at target cell $i$. A direct evaluation costs $O(N^2)$ per RHS call.

In two dimensions, the demag field can be written as a convolution:

$$\begin{bmatrix} h_x \\ h_y \\ h_z \end{bmatrix} = \begin{bmatrix} N_{xx} & N_{xy} & N_{xz} \\ N_{yx} & N_{yy} & N_{yz} \\ N_{zx} & N_{zy} & N_{zz} \end{bmatrix} * \begin{bmatrix} m_x \\ m_y \\ m_z \end{bmatrix},$$

or equivalently,

$$h_\alpha(i,j) = \sum_{m,n} \sum_{\beta \in \{x,y,z\}} N_{\alpha\beta}(i-m, j-n) m_\beta(m,n).$$

Using the convolution theorem,

$$\widehat{h}_\alpha(k_x,k_y) = \sum_{\beta \in \{x,y,z\}} \widehat{N}_{\alpha\beta}(k_x,k_y)\widehat{m}_\beta(k_x,k_y), \qquad h_{\text{demag}} = \text{IFFT}\left(\widehat{h}_{\text{demag}}\right),$$

which reduces the demag cost from $O(N^2)$ to $O(N \log N)$. This matters because demag is inside the CVODE RHS: its cost is paid during adaptive stepping, Newton iterations, and Krylov linear solves.

### FFT Demag Pipeline

The implementation separates geometry-dependent work from state-dependent work:

$$\texttt{Demag_Init:} \qquad N \to \widehat{N} \qquad \text{computed once,}$$

$$\texttt{Demag_Apply:} \qquad m \to \widehat{m} \to \widehat{h}_{\text{demag}} = \widehat{N}\widehat{m} \to h_{\text{demag}} \qquad \text{computed every RHS call.}$$

The demag tensor depends only on grid geometry and cell parameters, so `Demag_Init` builds the Newell-style tensor once and stores its Fourier-space coefficients on the GPU. `Demag_Apply` performs the state-dependent path shown in Fig. 3.

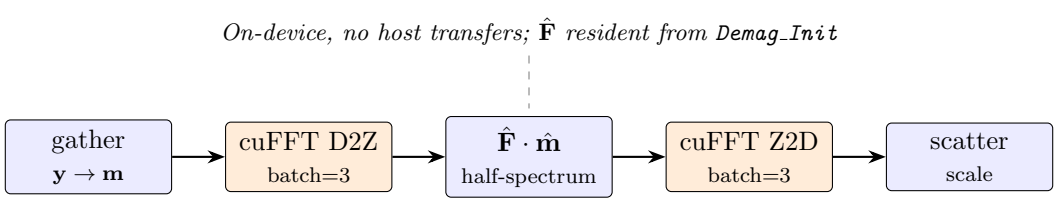


Figure 3: GPU-resident demag pipeline. The 9-component half-spectrum tensor $\hat{\mathbf{F}}$ is computed once in `Demag_Init` and reused in every RHS call.

The gather step packs the SoA state $y = [m_x|m_y|m_z]$ into real FFT buffers. The three components are transformed as a batch using `cufftExecD2Z`; the half-spectrum is multiplied by the precomputed tensor; the result is transformed back with `cufftExecZ2D`; and the real-space demag field is scattered back to SoA layout with scale

$$\texttt{DEMAG_STRENGTH}/(n_x n_y).$$

Because the fields are real-valued, the D2Z/Z2D path stores only

$$n_y \left( \frac{n_x}{2} + 1 \right)$$

complex values. This uses Hermitian symmetry to reduce frequency-domain storage and memory traffic. The full `Demag_Apply` path stays on the GPU: magnetization and demag fields are not copied through host memory inside the RHS.

## Direct Convolution Baseline

We implemented a direct convolution baseline with the same CVODE integration structure and the same demag tensor. For the head-on transition case,

$$n_x = 768, \qquad n_y = 256, \qquad n_g = 256,$$

so the demag grid contains

$$N = 256 \times 256 = 65{,}536$$

magnetic cells and

$$3N = 196{,}608$$

ODE unknowns. A single direct demag evaluation requires

$$N^2 = 65{,}536^2 = 4{,}294{,}967{,}296$$

source-target interactions before the $3 \times 3$ tensor-vector work.

The direct-vs-FFT comparison used the same head-on setup, `DEMAG_STRENGTH=1.0`, `DEMAG_THICK=1.0`, $T_{\text{total}} = 1000$, and tolerance $10^{-4}$.

| Method | Complexity | Time (ms) | nfe | nst |
|---|---|---|---|---|
| FFT demag, GPU-resident | $O(N \log N)$ | **17,594.191** | 23,300 | 16,715 |
| Direct demag | $O(N^2)$ | 2,108,222.750 | 23,419 | 16,603 |

The FFT path is

$$\frac{2{,}108{,}222.750}{17{,}594.191} \approx 119.8\times$$

faster end-to-end. The two runs have similar CVODE statistics, so the difference comes mainly from demag evaluation rather than a different integration path.

## FFT Optimization Path

### Batched cuFFT

The first FFT version issued six cuFFT calls per RHS evaluation: three forward transforms for $m_x, m_y, m_z$, and three inverse transforms for $h_x, h_y, h_z$. We replaced this with `cufftPlanMany` using `batch=3`. The three components are stored contiguously, so the forward transforms become one batched call and the inverse transforms become another. This reduces cuFFT calls from 6 to 2 per RHS evaluation without changing the mathematics.

### Real-to-Complex FFT Path

The magnetization and demag fields are real-valued, so their Fourier spectra have Hermitian symmetry. The D2Z/Z2D path stores only

$$n_y \left( \frac{n_x}{2} + 1 \right)$$

complex values instead of $n_x n_y$. For a $512 \times 512$ demag grid,

$$n_{\text{half}} = 512(512/2 + 1) = 131{,}584,$$

which is 50.2% of the full complex spectrum. Tensor-spectrum storage decreases from about 37.7 MB in the Z2Z implementation to about 18.9 MB in the D2Z/Z2D implementation.

| FFT path | Time (ms) | nst | nfe | nli | ncfn | netf |
|---|---|---|---|---|---|---|
| Z2Z batched | 32,395.908 | 23,551 | 29,893 | 85,263 | 60 | 5 |
| D2Z/Z2D batched | **29,117.648** | 23,982 | 30,027 | 85,150 | 112 | 5 |

The end-to-end speedup is

$$\frac{32{,}395.908}{29{,}117.648} \approx 1.11 \times .$$

Nsight kernel time decreases from 25.88 s to 22.85 s, about 1.13×.

| Component | Z2Z time (s) | D2Z/Z2D time (s) | Change |
|---|---|---|---|
| `regular_fft` | 3.91 | 2.17 | −44.5% |
| `vector_fft` family | 2.66 | 2.45 | −7.9% |
| `multiply_kernel` | 1.83 | 0.82 | −55.5% |
| pack/gather + unshift/scatter | 0.33 | 0.24 | −27.3% |
| FFT pipeline total | 8.73 | 5.56 | −36.3% |

The benefit is size-dependent. On smaller $256 \times 256$ FFT grids, launch and library overhead hide much of the half-spectrum advantage; on $512 \times 512$, the memory-traffic reduction becomes visible.

### Removing Fused Linear Combination

Following the adaptive Gram–Schmidt strategy from Optimization 1, the FFT path also disables the custom fused linear-combination kernel and falls back to `N_VScale` and `N_VLinearSum`. For $K \geq 5$ the fused kernel streams many full Krylov vectors per thread, producing high memory traffic and poor cache behavior; the simpler operations keep more kernels in the trace but each touches fewer full vectors and is more bandwidth-friendly.

### Magnetization Normalization

The LLG equation assumes $|m| = 1$. In the stabilized FFT path, magnetization is normalized before RHS evaluation:

$$m \leftarrow \frac{m}{\|m\|}.$$

This improves nonlinear robustness at small cost; in the larger D2Z/Z2D profile, `normalize_m_kernel` accounts for about 1.4% of GPU kernel time.

For the later irregular `i4` experiment, we also tested deferred normalization: instead of normalizing inside every RHS evaluation, normalization is applied once at the time-step/output level. This reduces repeated kernel launches and global-memory passes through the magnetization vector.

### Krylov Dimension Sweep

We tuned the GMRES Krylov dimension. Larger Krylov spaces can reduce iteration count, but increase orthogonalization, dot products, vector storage, and memory traffic.

| Krylov dim | Time (ms) | nfe | nli |
|---|---|---|---|
| 0 | 17,263 | 35,678 | 108,296 |
| 5 | **8,657** | 17,895 | 51,846 |
| 10 | 13,482 | 28,709 | 77,555 |
| 15 | 14,310 | 29,441 | 80,203 |
| 20 | 15,710 | 33,657 | 90,342 |
| 30 | 13,844 | 29,159 | 83,766 |

The best point is $K = 5$:

$$\frac{17{,}263}{8{,}657} \approx 1.99\times$$

faster than $K = 0$. The trend is not monotonic because runtime balances convergence improvement against vector-operation cost.

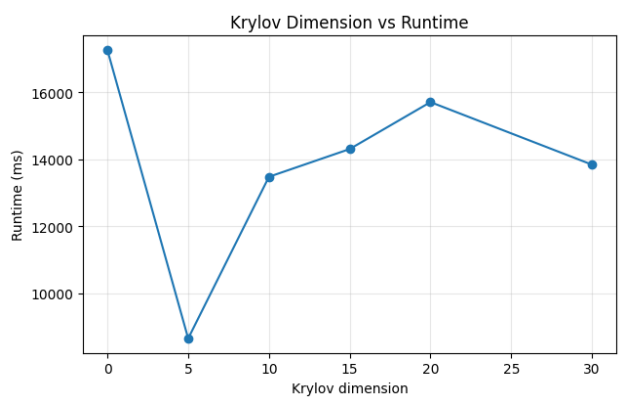


Figure 4: Runtime as a function of Krylov dimension. A moderate Krylov dimension gives the best end-to-end time because it balances convergence rate against vector-operation and memory-traffic cost.

## Profiling After FFT Optimizations

After FFT removes the $O(N^2)$ demag bottleneck, runtime is distributed across cuFFT, SUNDIALS vector operations, reductions, JTV, preconditioning, and synchronization.

For the larger D2Z/Z2D profile, the dominant CUDA API and GPU kernel items are:

| Kernel/API item | Time share | Total time | Instances |
|---|---|---|---|
| `cudaStreamSynchronize` | 77.4% API | 23.67 s | 509,144 |
| `cudaLaunchKernel` | 15.0% API | 4.59 s | 2,253,869 |
| `cudaMemcpyAsync` | 6.4% API | 1.95 s | 994,237 |
| `linearSumKernel` | 16.3% GPU | 3.73 s | 687,473 |
| `dotProdKernel` | 12.0% GPU | 2.73 s | 371,870 |
| `jtv_kernel` | 10.7% GPU | 2.44 s | 85,711 |
| `regular_fft` | 9.5% GPU | 2.17 s | 59,969 |
| `apply_P_kernel` | 7.4% GPU | 1.68 s | 115,687 |
| `vector_fft_c2r` | 5.4% GPU | 1.23 s | 29,980 |
| `vector_fft_r2c` | 5.3% GPU | 1.22 s | 29,989 |
| `multiply_kernel` | 3.6% GPU | 0.82 s | 29,980 |
| `f_kernel_unified` | 2.8% GPU | 0.63 s | 29,980 |
| `normalize_m_kernel` | 1.4% GPU | 0.31 s | 29,980 |

The custom RHS kernel is 2.8% of GPU time, and the spectral multiply is 3.6%. The main costs are SUNDIALS vector operations, reductions, Krylov work, cuFFT, and synchronization across solver stages.

The gather/scatter kernels are small but necessary layout-conversion steps. CVODE stores the solution in a single SoA-style `N_Vector`, while cuFFT expects contiguous real input buffers for each transformed component. Gather packs $m_x, m_y, m_z$ into the batched FFT buffer; scatter writes the inverse transformed demag field back into the solver layout.

## FFT Scaling with Problem Size

We evaluate the optimized D2Z/Z2D FFT-demag solver by increasing problem size with

$$n_x = 3n_y, \qquad n_g = n_x/3 = n_y,$$

so the demag FFT grid is square:

$$n_g \times n_y = n_y \times n_y.$$

All runs use `DEMAG_STRENGTH=2.0`, `DEMAG_THICK=1.0`, `KRYLOV_DIM=5`, $T_{\text{total}} = 1000$, and tolerance $10^{-4}$.

| $n_x$ | $n_y$ | FFT grid | $N$ cells | Time (ms) | nfe | nli |
|---|---|---|---|---|---|---|
| 96 | 32 | $32^2$ | 1,024 | 12,225.644 | 41,788 | 90,773 |
| 192 | 64 | $64^2$ | 4,096 | 9,702.420 | 28,897 | 88,664 |
| 288 | 96 | $96^2$ | 9,216 | 9,457.688 | 27,541 | 73,354 |
| 384 | 128 | $128^2$ | 16,384 | 9,980.323 | 27,280 | 77,443 |
| 576 | 192 | $192^2$ | 36,864 | 19,747.936 | 44,311 | 136,238 |
| 768 | 256 | $256^2$ | 65,536 | 13,405.807 | 28,998 | 79,206 |
| 1152 | 384 | $384^2$ | 147,456 | 28,091.684 | 36,435 | 102,853 |
| 1536 | 512 | $512^2$ | 262,144 | 29,970.914 | 30,530 | 84,173 |
| 2304 | 768 | $768^2$ | 589,824 | 96,161.867 | 38,668 | 135,059 |
| 3072 | 1024 | $1024^2$ | 1,048,576 | 187,346.719 | 41,053 | 144,013 |

This is an end-to-end adaptive CVODE run, not a fixed number of FFT calls, so runtime depends on both per-RHS cost and solver-selected work. Small grids are nearly flat because fixed CVODE overhead, launches, synchronization, and SUNDIALS vector operations dominate. Beyond the $512^2$ FFT grid, runtime grows more sharply.

From $512^2$ to $1024^2$, the number of magnetic cells increases by $4\times$, while solver work increases from 30,530 to 41,053 RHS evaluations and from 84,173 to 144,013 linear iterations. The slowdown is therefore a combination of larger per-operation memory traffic and more adaptive solver work. This is a regime change: beyond $512^2$, cuFFT, spectral multiplication, and SUNDIALS vector kernels become bandwidth-limited enough to dominate runtime growth.

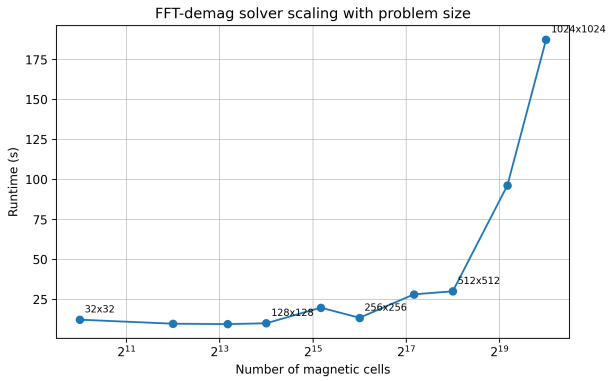


Figure 5: Runtime scaling of the FFT-based demagnetization solver.

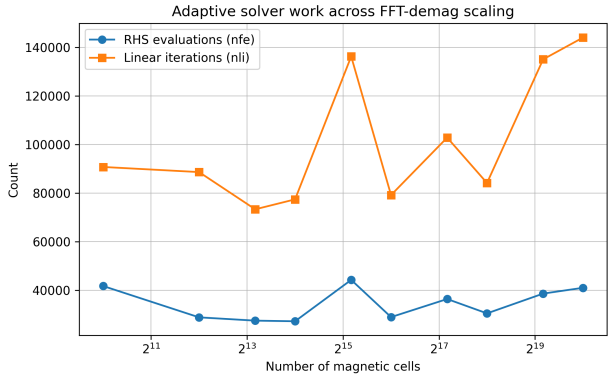


Figure 6: Adaptive solver work across problem sizes. Runtime is not only a function of FFT grid size; it also depends on RHS evaluations and linear iterations chosen by CVODE.

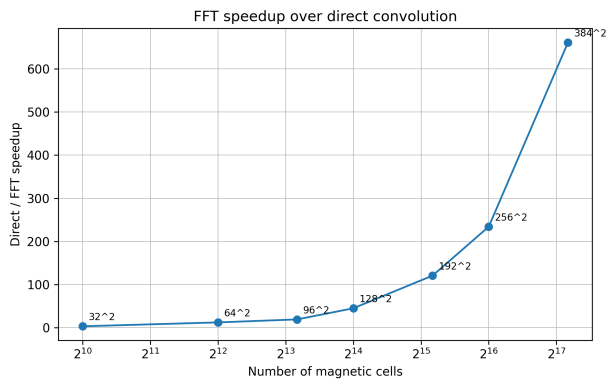


Figure 7: End-to-end speedup of the FFT demagnetization path over direct convolution. The speedup increases rapidly with problem size because the direct method scales as $O(N^2)$, while the FFT method scales much closer to $O(N \log N)$.

The direct-vs-FFT scaling confirms the algorithmic reason for using FFT demag. At small sizes, fixed solver overhead is still visible; as the number of cells increases, the $O(N^2)$ direct method grows much faster than the FFT method. The speedup rises from a few times at the smallest grid to hundreds of times at larger grids.

**Conclusion.** FFT demag is required for practical long-range magnetostatic simulation. After the $O(N^2)$ bottleneck is removed, runtime still grows once the working set exceeds the cache-friendly regime, but the FFT path remains far more scalable than direct convolution.

## Irregular FFT Geometry Cases

We use four irregular FFT cases to test different physical geometries and initial conditions. All cases keep the CVODE state in SoA layout,

$$[m_x \text{ for all cells}][m_y \text{ for all cells}][m_z \text{ for all cells}],$$

and all use the same GPU-resident FFT demag pipeline. The main difference is how the active magnetic region is defined and how the initial magnetization is assigned.

**Grouping of cases.** The four configurations fall into two physical categories:

- **Square-hole film (i2, i4):** a continuous magnetic film with a geometric defect (hole). The dynamics are driven by defect-induced symmetry breaking.
- **Confined active region (i5, i6):** a finite magnetic region embedded in an inactive background. The dynamics are boundary-dominated rather than defect-driven.

**i2: square hole with uniform active material.** The `i2` case uses a centered square hole inside an otherwise active periodic film. The geometry is represented by a dense SoA mask `ymsk[3*ncell]`:

$$\texttt{ymsk} = 1 \quad \text{on active cells,} \qquad \texttt{ymsk} = 0 \quad \text{inside the hole.}$$

The square hole is centered at

$$(c_x, c_y) = (\texttt{HOLE_CENTER_X_FRAC}(n_g - 1), \texttt{HOLE_CENTER_Y_FRAC}(n_y - 1)),$$

with half-side length

$$\texttt{halfside} = \texttt{HOLE_RADIUS_FRAC_Y} \cdot n_y.$$

A cell is inside the hole if

$$|i - c_x| \le \texttt{halfside} \quad \text{and} \quad |j - c_y| \le \texttt{halfside}.$$

The RHS still launches over the full structured grid. Inactive cells are handled only by masking the output derivative:

$$\dot{m}_\alpha(i) = \texttt{ymsk}_\alpha(i)\, f_\alpha(i).$$

Thus hole cells have $\dot{m} = 0$ and remain frozen at their initial value. Neighbor reads require no special branch: a neighbor inside the hole already has $m = 0$.

Active cells are initialized uniformly from

$$m = (\texttt{INIT_MX}, \texttt{INIT_MY}, \texttt{INIT_MZ}) = (1.0, -0.0175, 0.0),$$

normalized to unit length. Hole cells are initialized to zero. This case isolates the effect of the square-hole defect: any nonuniform structure that develops is generated by the interaction between the hole boundary, exchange, anisotropy, demag, and LLG dynamics.

**i4: square hole with asymmetric tail initialization.** The `i4` case keeps the same square-hole mask as `i2`, but changes the initial condition to drive stronger asymmetric dynamics around the hole. The active region is split horizontally at

$$j_{\text{split}} = \frac{7n_y}{8}.$$

Cells above this line form a top stripe initialized approximately along $-x$:

$$m = (-1, 0, 0).$$

Cells below the split are initialized along $+x$, with opposite small $m_y$ perturbations on the left and right:

$$m = (+1, -m_{\text{tail}}, 0) \quad \text{for } i < n_g/2,$$
$$m = (+1, +m_{\text{tail}}, 0) \quad \text{for } i \ge n_g/2.$$

This initialization is designed to seed tail or spike-domain structures around the hole, which are otherwise difficult to trigger from a nearly uniform state.

The vector is normalized after assignment, and hole cells remain exactly zero.

This creates a top–bottom reversal together with left–right asymmetry. Compared with the nearly uniform `i2` case, `i4` contains stronger gradients near the square hole and is a better test for tail or spike-like domain formation.

For visualization, we use the diagnostic map

$$\text{MAP} = [m_x(j,i) - m_x(j-1,i)] - [m_y(j,i) - m_y(j,i-1)] \approx \partial_y m_x - \partial_x m_y.$$

This highlights local rotation, chirality, and tail structure more clearly than raw component plots alone.

In `i4`, we also test deferred normalization: instead of normalizing inside every RHS evaluation, normalization is applied once per time-step/output interval. This removes repeated full-vector passes through the magnetization state. Since profiling shows that runtime is dominated by many solver-driven kernels and synchronization points, reducing normalization frequency targets the actual observed overhead rather than only the physics kernel.

**i5: centered active rectangle.** The `i5` case reverses the usual hole setup. Instead of an active film with an inactive hole, only a centered rectangle is active and the exterior is masked out. The full CVODE/FFT grid remains dense and structured, but

$$\texttt{ymsk} = 1 \quad \text{only inside the centered rectangle,}$$

and

$$\texttt{ymsk} = 0 \quad \text{outside it.}$$

The rectangle has width and height

$$\texttt{OUTER_W_FRAC} \cdot n_g, \qquad \texttt{OUTER_H_FRAC} \cdot n_y,$$

with default values

$$\texttt{OUTER_W_FRAC} = \texttt{OUTER_H_FRAC} = 0.5.$$

Equivalently, a cell is active if it lies inside the centered rectangle,

$$|i - c_x| \le \frac{1}{2}\texttt{OUTER_W_FRAC}\, n_g, \qquad |j - c_y| \le \frac{1}{2}\texttt{OUTER_H_FRAC}\, n_y.$$

Active cells are initialized uniformly from

$$m = (\texttt{INIT_MX}, \texttt{INIT_MY}, \texttt{INIT_MZ}),$$

normalized to unit length; exterior cells are initialized to zero and remain frozen by the mask. This case represents a confined magnetic island embedded in a nonmagnetic background. It stresses the boundary between active and inactive material while preserving the dense grid required by FFT demag.

**i6: centered circular/elliptical active region with compact execution.** The `i6` case changes both the geometry and the execution strategy. The active region is a centered ellipse,

$$\frac{(i - c_x)^2}{r_x^2} + \frac{(j - c_y)^2}{r_y^2} \le 1,$$

with

$$r_x = \texttt{ACTIVE_RX_FRAC} \cdot n_g, \qquad r_y = \texttt{ACTIVE_RY_FRAC} \cdot n_y.$$

The default values are

$$\texttt{ACTIVE_RX_FRAC} = \texttt{ACTIVE_RY_FRAC} = 0.25,$$

so on a square grid this becomes a centered circular active region. Cells outside the ellipse are inactive, initialized to zero, and frozen.

Unlike `i2`–`i5`, `i6` uses compact active-cell execution. The host builds two index lists:

$$\texttt{d_active_ids}[0 : n_{\text{active}}], \qquad \texttt{d_inactive_ids}[0 : n_{\text{inactive}}].$$

The RHS, JTV, preconditioner setup, and preconditioner apply kernels launch one thread per active cell:

$$\texttt{tid} \to \texttt{cell} = \texttt{d_active_ids[tid]}.$$

Before each active-cell RHS call, a separate kernel clears the derivative on inactive cells:

$$\texttt{zero_inactive_kernel} \quad \to \quad \texttt{compact RHS over active cells.}$$

The FFT demag pipeline still uses dense grid buffers. Although the local kernels operate only on active cells, the convolution requires a regular lattice. Inactive cells remain zero and therefore do not contribute to the demagnetization field.

**Comparison.** The four cases represent different physical scenarios, not just different code paths:

- `i2`: square-hole defect with nearly uniform active material.
- `i4`: square-hole defect with asymmetric tail/spike-driving initialization.
- `i5`: centered active rectangle surrounded by inactive background.
- `i6`: centered circular/elliptical active region with compact active-cell execution.

The implementation tradeoff is between dense masking and compaction. Dense `ymsk` execution keeps memory access regular and matches the FFT layout, but still launches local work over inactive cells. Compact execution reduces local RHS work, but introduces index indirection and still requires dense FFT buffers for demag. This is the central irregular-geometry tradeoff in the FFT solver: the physical material can be irregular, but the long-range convolution still needs a structured grid.

## Irregular Polycrystal FFT Case: i3

The `i3` case is the most complete irregular configuration in the project: it combines local stencil work, FFT demag, irregular masking, heterogeneous material directions, and the CVODE/SPGMR solver stack on a 2D Voronoi-polycrystal film with dead-grain holes, soft boundaries, per-cell easy axes, per-cell DMI directions, and long-range dipolar coupling computed by batched cuFFT.

### Physical model

We integrate the simplified Landau–Lifshitz–Gilbert equation

$$\frac{d\mathbf{m}}{dt} = \gamma\, \mathbf{m} \times \mathbf{h}_{\text{eff}} + \alpha\big(\mathbf{h}_{\text{eff}} - (\mathbf{m}\cdot\mathbf{h}_{\text{eff}})\,\mathbf{m}\big), \qquad |\mathbf{m}| = 1, \tag{1}$$

on a 2D periodic grid of $n_g \times n_y$ cells, with

$$\mathbf{h}_{\text{eff}}(\mathbf{r}) = \underbrace{c_{\text{ex}} \sum_{\mathbf{r}'\in\mathcal{N}(\mathbf{r})} \mathbf{m}(\mathbf{r}')}_{\text{exchange (5-pt stencil)}} + \underbrace{\big[c_K(\mathbf{m}\cdot\hat{\mathbf{e}}_K) + c_a\big]\,\hat{\mathbf{e}}_K}_{\text{uniaxial anisotropy}} + \underbrace{c_b\,\hat{\mathbf{e}}_D\big(\mathbf{m}_L + \mathbf{m}_R\big)\cdot\hat{\mathbf{x}}}_{\text{DMI}} + \underbrace{\mathbf{h}_{\text{dmag}}(\mathbf{r})}_{\text{long-range dipolar}}. \tag{2}$$

The geometry is a Voronoi polycrystal with 72 grains (Fig. 8). A random 16% of grains are killed, producing polygonal holes. A soft tanh weight $w(\mathbf{r}) \in [0,1]$ is computed once on the host and uploaded to the GPU. Cells with $w \approx 0$ remain frozen at $\mathbf{m} = 0$, and demag is computed on the windowed magnetization $w\mathbf{m}$ so that holes do not contribute to the dipolar field.

Each grain has a randomly drawn easy-axis direction $\hat{\mathbf{e}}_K^{(g)}$ and DMI direction $\hat{\mathbf{e}}_D^{(g)}$. Both are stored per cell in SoA arrays `d_msk[3*ncell]` and `d_nsk[3*ncell]`. This produces coherent behavior within a grain and discontinuous changes across grain boundaries.

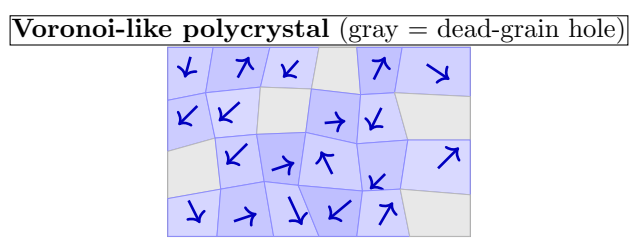


Figure 8: Schematic of the irregular polycrystal geometry. Each active grain has one easy-axis direction, shown by the arrow; gray grains are killed and become holes with softened boundaries in the simulation.

### Why this case stresses the GPU pipeline

The `i3` case exercises all three parts of the implementation:

- The 5-point exchange term is a stencil/convolution pattern.
- The dipolar field is a global interaction reduced from $O(N^2)$ to $O(N \log N)$ by FFT.
- CVODE/SPGMR introduces many NVector operations, reductions, and synchronization points.
- Holes and soft boundaries test dense masked execution on an otherwise structured FFT grid.

### Local RHS Kernel Optimizations

The polycrystal RHS is evaluated by one fused CUDA kernel, `f_kernel_unified_polycrystal_tiled`. For each cell, it computes $\mathbf{h}_{\text{eff}}$ from Eq. 2, applies the LLG cross-and-damping form, and gates the result by the soft weight $w$. Beyond the SoA layout and $16 \times 8$ block shape established in the regular-grid baseline, the heterogeneous, masked, soft-boundary domain in i3 motivates four additional kernel-level optimizations.

**Tiled shared-memory stencil.** A naive 5-point stencil reads, per output cell,

$$13 \text{ doubles from global memory} = \underbrace{4 \times 3}_{4 \text{ neighbors} \times 3 \text{ components}} + 1\,(w_{\text{self}}).$$

With $16 \times 8$ blocks, each $x$-neighbor read by one thread is the self-read of a neighboring thread. The kernel therefore loads a $(B_x + 2) \times (B_y + 2)$ halo into shared memory once per block and then reads the stencil from `__shared__` memory.

```
__shared__ sunrealtype tile[3 * (BLOCK_X+2) * (BLOCK_Y+2)];
#define TILE(a, sx, sy) tile[(a)*SX*SY + (sy)*SX + (sx)]

// Every thread loads its own (w * m) into the interior of the tile.
LOAD_WM(gx, gy, tx+1, ty+1);

// Block-edge threads cooperate to load the halo.
if (tx == 0)            LOAD_WM(wrap_x(gx-1,ng), gy, 0,          ty+1);
if (tx == BLOCK_X-1)    LOAD_WM(wrap_x(gx+1,ng), gy, BLOCK_X+1, ty+1);
if (ty == 0)            LOAD_WM(gx, wrap_y(gy-1,ny), tx+1,      0);
if (ty == BLOCK_Y-1)    LOAD_WM(gx, wrap_y(gy+1,ny), tx+1,      BLOCK_Y+1);

__syncthreads();

// 5-point stencil read: now from shared memory.
const sunrealtype lx1 = TILE(0, sx-1, sy);
const sunrealtype rx1 = TILE(0, sx+1, sy);
const sunrealtype ux1 = TILE(0, sx,   sy-1);
const sunrealtype dx1 = TILE(0, sx,   sy+1);
// ...same for components 2, 3
```

Listing 2: Tiled stencil load (paraphrased from `f_kernel_unified_polycrystal_tiled`).

The shared-memory footprint is $3 \times 18 \times 10 \times 8\,\text{B} = 4.3$ KB per block, well below the 99 KB per-block budget on `sm_89`. The stencil load count drops from about 13 global doubles per cell to about 3, consistent with the measured $\sim 28\,\mu$s RHS kernel time at $n_{\text{cell}} = 65{,}536$.

**Fused window-and-gather.** The first version materialized $\mathbf{y}_{\text{eff}} = w\mathbf{y}$ as a separate $3n_{\text{cell}}$ buffer, written by `apply_weight_kernel` and then read by both the demag pipeline and the RHS stencil:

$$\text{normalize} \to \underbrace{\text{write } \mathbf{y}_{\text{eff}}}_{1.5\,\text{MB}} \to \text{demag reads } \mathbf{y}_{\text{eff}} \to \text{RHS reads } \mathbf{y}_{\text{eff}}.$$

We removed this buffer by pushing the multiplication into the consumers:

- `gather_real_kernel_windowed` reads $\mathbf{y}$ and $w$, multiplies, and writes the FFT input `d_m_real`.
- The RHS stencil applies $w$ during the shared-memory load.

  At $n_{\text{cell}} = 65{,}536$, this saves

$$3 \times 65{,}536 \times 8\,\text{B} = 1.5\ \text{MB}$$

per RHS evaluation, or roughly 60 GB across $\sim 4 \times 10^4$ RHS evaluations.

**Branch-free normalization.** Earlier versions used `if (w[cell] < 1e-3) return;` in `normalize_m_kernel`. Because $w$ varies smoothly across soft boundaries, warps crossing a boundary diverged. We replaced the branch with

$$\mathbf{m} \leftarrow \frac{\mathbf{m}}{|\mathbf{m}| + \varepsilon}, \qquad \varepsilon = 10^{-3}.$$

This keeps hole cells fixed:

$$|\mathbf{m}| = 0 \Rightarrow \frac{0}{0+\varepsilon} = 0,$$

and keeps active cells near unit length:

$$|\mathbf{m}| \approx 1 \Rightarrow \frac{\mathbf{m}}{1+\varepsilon} \approx 0.999\,\mathbf{m}.$$

The kernel then runs branch-free over all cells.

**Read-only cache and launch bounds.** The arrays $w$, `msk`, `nsk`, and `h_dmag` are read-only inside the RHS. Their loads use `__ldg`:

```
const sunrealtype mskx   = __ldg(&msk[mx_i]);
const sunrealtype nskx   = __ldg(&nsk[mx_i]);
const sunrealtype hd1    = __ldg(&h_dmag[mx_i]);
const sunrealtype w_self = __ldg(&w[cell]);
```

The kernel is also annotated with

```
__launch_bounds__(BLOCK_X * BLOCK_Y, 2)
```

to keep at least two blocks resident per SM and avoid excessive register growth.

### FFT demag with windowed magnetization

The i3 case uses the same GPU-resident FFT demag pipeline as Fig. 3. The difference is that the FFT input is the windowed magnetization

$$\mathbf{m}_{\text{eff}} = w\mathbf{m},$$

so dead grains and soft boundaries do not leak into the dipolar field:

$$\mathbf{h}_{\text{dmag}}(\mathbf{r}) = \sum_{\mathbf{r}'} \mathbf{F}(\mathbf{r} - \mathbf{r}') \cdot \mathbf{m}_{\text{eff}}(\mathbf{r}'). \tag{3}$$

This keeps the FFT path dense and structured while allowing the physical geometry to contain holes and softened grain boundaries.

### Solver-layer reuse and stream topology

The i3 case applies the three solver-level optimizations developed earlier on the regular grid without modification: the custom `multi_dot_kernel` override of `nvdotprodmulti` under Classical Gram–Schmidt; the analytic $3 \times 3$ block-Jacobi preconditioner with separated $\mathbf{J}$ build and $\mathbf{P}^{-1}$ construction; and the analytic Jacobian–vector product covering exchange, anisotropy, and DMI through the same 4-neighbor stencil. The demag contribution is omitted from JTV because adding a second FFT pipeline per Jv evaluation would not pay off relative to the preconditioned Krylov work it would save. The fused linear-combination override is disabled, again following the adaptive Gram–Schmidt strategy from Optimization 1.

**CUDA streams.** The RHS wrapper uses two CUDA streams (Fig. 9). Normalization runs on `stream_norm`; the demag pipeline runs on `stream_demag`; the final tiled RHS kernel launches after both complete. The events serialize the two streams within a single RHS call, so this is not true overlap. The benefit is that the long cuFFT pipeline does not occupy the default stream used by SUNDIALS NVector operations.

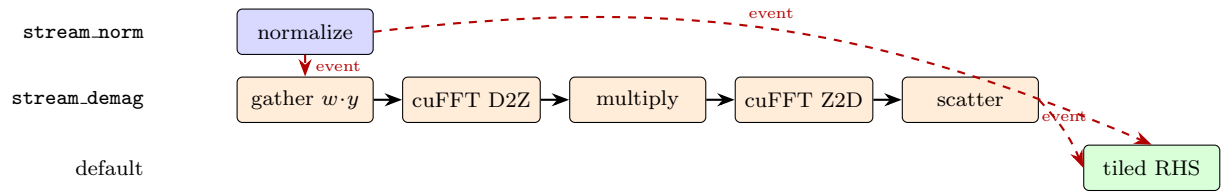


Figure 9: Stream and event topology of one RHS call. The streams are sequenced by events because demag depends on normalized $\mathbf{y}$; the main benefit is decoupling cuFFT from the default stream used by SUNDIALS NVector operations.

### Profiling

We profiled a full `run-demag` run with Nsight Systems at

$$n_g = 256, \quad n_y = 256, \quad n_{\text{cell}} = 65{,}536, \quad n_{\text{eq}} = 196{,}608,$$

with $T = 1000$, KRYLOV $= 5$, and Classical Gram–Schmidt. Total GPU kernel time was **30.6 s**.

Table 2: Kernel-time breakdown from Nsight Systems. Bold entries are kernels we wrote; the rest are SUNDIALS or cuFFT kernels.

| **Kernel** | **Time** | **%** | **Layer** |
|---|---|---|---|
| `linearSumKernel` (SUNDIALS) | 6.13 s | 20.0% | solver / NVector |
| `regular_fft` (cuFFT, fwd) | 3.03 s | 9.9% | long-range |
| `dotProdKernel` (SUNDIALS) | 2.49 s | 8.1% | solver / NVector |
| `jtv_kernel` | **2.41 s** | **7.9%** | solver / physics |
| `apply_P_kernel` | **2.11 s** | **6.9%** | solver |
| `vector_fft_r2c` (cuFFT) | 1.84 s | 6.0% | long-range |
| `scaleKernel` (SUNDIALS) | 1.71 s | 5.6% | solver / NVector |
| `vector_fft_c2r` (cuFFT) | 1.69 s | 5.5% | long-range |
| `wL2NormSquareKernel` (SUNDIALS) | 1.67 s | 5.5% | solver / NVector |
| `divKernel` (SUNDIALS) | 1.40 s | 4.6% | solver / NVector |
| `f_kernel_unified_polycrystal_tiled` | **1.31 s** | **4.3%** | local RHS |
| `scaleAddMultiKernel` (SUNDIALS) | 1.23 s | 4.0% | solver / NVector |
| `multiply_kernel` | **1.16 s** | **3.8%** | long-range |
| `prodKernel` (SUNDIALS) | 0.53 s | 1.7% | solver / NVector |
| `normalize_m_kernel` | **0.51 s** | **1.7%** | local |
| `setConstKernel` (SUNDIALS) | 0.41 s | 1.3% | solver / NVector |
| `gather_real_kernel_windowed` | **0.33 s** | **1.1%** | long-range |
| others (build_J, scatter, abs, inv, etc.) | ∼0.5 s | ∼2% | various |

Summing the SUNDIALS NVector kernels gives about **55%** of total GPU time. cuFFT accounts for about **21%**. The RHS, normalization, gather/scatter, and spectral multiply together account for about **12%**; JTV and preconditioning account for another **12%**.

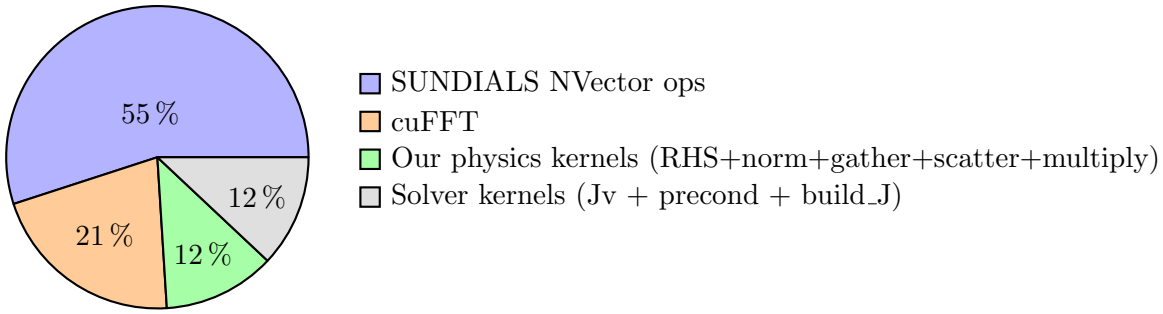


Figure 10: GPU-time breakdown by category. The local RHS kernel alone is only 4.3% of GPU time.

The numbers lead to several concrete conclusions.

**The tiled RHS is not the main target.** The RHS kernel runs in about 32 $\mu$s per call, processing $n_{\text{eq}} = 196{,}608$ doubles of $\mathbf{y}$ plus roughly $5n_{\text{cell}}$ doubles of metadata and shared-memory stencil reads. The effective DRAM throughput is about 70–80 GB/s, consistent with a bandwidth-limited double-precision 2D stencil on `sm_89`. Further RHS work would require changing precision or physics.

**cuFFT dominates the long-range layer.** The cuFFT kernels `regular_fft`, `vector_fft_r2c`, and `vector_fft_c2r` total about 6.6 s, much more than `multiply_kernel` and gather/scatter. The spectral multiply is not the long-range bottleneck; the FFTs are.

**SUNDIALS vector operations form the floor.** `linearSumKernel` alone is 20% of GPU time and is launched hundreds of thousands of times. These operations occur inside CVODE's Newton, BDF, and SPGMR update sequence. Reducing them would require modifying SUNDIALS or capturing dynamic solver control flow with CUDA Graphs.

**Visible solver kernels are not necessarily bad kernels.** `apply_P_kernel` and `jtv_kernel` appear high in the profile because they replace more expensive work elsewhere: preconditioning reduces linear-solver cost, and analytic JTV avoids finite-difference RHS evaluations.

**Launch overhead remains visible.** The CUDA API trace reports 2,624,593 `cudaLaunchKernel` calls, totaling 4.99 s, about 1.9 $\mu$s per launch. This is launch latency, not GPU work. The count is set mainly by SUNDIALS' internal kernel granularity.

### Remaining Headroom

The remaining optimization targets are mostly outside the local physics kernel.

Table 3: Potential further optimizations and expected returns.

| Optimization | Best-case savings | Cost / risk |
|---|---|---|
| Single-precision build | cuFFT 21% $\rightarrow$ 12%; RHS may halve | Must revalidate $\lvert\mathbf{m}\rvert = 1$ dynamics; BDF may need lower order |
| CUDA Graphs around Newton/GMRES loops | Reduce $\sim$5 s launch overhead | Hard to capture SUNDIALS' dynamic control flow; version-specific |
| Replace more SUNDIALS NVector ops | Part of the 55% NVector time | Fragile; risk of silent solver errors |
| Hierarchical matrix / FMM for demag | Reduce cuFFT share | Major new implementation; accuracy revalidation |
| Add demag contribution to JTV | Possibly reduce Krylov work | Requires a second FFT pipeline per Jv; likely near break-even |
| Persistent-thread RHS kernel | 10–20% of RHS only | RHS is 4.3%; total impact $< 1\%$ |
| Texture-fetch Newell tensor | Marginal | Negligible at this size |

At this point, the local and physics kernels are not the limiting factor. The largest GPU cost is SUNDIALS NVector bookkeeping, and the largest CPU-side cost is kernel-launch orchestration across millions of small solver operations. Further large gains would require integrator-level changes rather than another round of RHS tuning.

### i3 Summary

The `i3` case shows the final performance hierarchy of the project: local kernel optimizations make the RHS fast, FFT makes long-range demag scalable, and solver-level overrides reduce some CVODE/SPGMR overhead. Once all three are in place, the binding cost is SUNDIALS' vector-operation and orchestration layer, not the local physics kernel.

## Work Distribution

This project was completed individually.

All components of the system, including CUDA kernel implementation, CVODE integration, FFT demagnetization, solver optimizations, and performance analysis, were designed and implemented by Qianru Wu.

Therefore, the contribution is: 100% (Qianru Wu)

## References


[1] D. B. Kirk and W.-M. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 4th ed., Morgan Kaufmann, 2022.
`https://www.oreilly.com/library/view/programming-massively-parallel/9780323984638/`

[2] A. C. Hindmarsh et al., "SUNDIALS: Suite of Nonlinear and Differential/Algebraic Equation Solvers," *ACM Transactions on Mathematical Software*, vol. 31, no. 3, pp. 363–396, 2005.
`https://github.com/LLNL/sundials`

[3] NVIDIA Corporation, *CUDA Toolkit Documentation*.
`https://docs.nvidia.com/cuda/`

[4] NVIDIA Corporation, *cuFFT Library User Guide*.
`https://docs.nvidia.com/cuda/cufft/`

[5] NVIDIA Corporation, *Nsight Systems and Nsight Compute*.
`https://developer.nvidia.com/nsight-tools`

[6] A. J. Newell, W. Williams, and D. J. Dunlop, "A Generalization of the Demagnetizing Tensor for Nonuniform Magnetization," *Journal of Geophysical Research: Solid Earth*, vol. 98, no. B6, 1993.

[7] "Incoherent Magnetic Switching of $L1_0$ FePt Grains,"
`https://ieeexplore.ieee.org/document/9269489`